

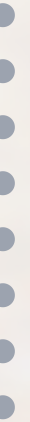
Loop Control Statements

Mastering Repetition in C Programming

 while

 do-while

 for





Introduction to Loop Control Statements



Loop control statements are used when a section of code may either be executed a **fixed number of times**, or **while some condition is true**.



while Loop

Repeats an action until condition returns false

Useful when loop count is unknown



do-while Loop

Condition checked after loop body is executed

Ensures loop runs at least once



for Loop

Frequently used for fixed number of iterations

Compact structure with initialization



Key Concept: Loops help avoid repetitive code by executing the same block multiple times with different values.





The while Loop



When a single statement or group of statements are to be executed **repeatedly** depending upon a certain test condition, the while statement is used.

Syntax:

```
while (test condition) {  
    body_of_the_loop;  
}
```



Test Condition

Expression that controls how long the loop keeps running



Body of the Loop

Statement or group of statements enclosed in braces



Entry-Control Loop

Also called entry-control loop as it controls execution at the entry point





Important: If condition initially is false, body of loop will never be executed.



while Loop Example

```
#include<stdio.h>
#include<conio.h>
void main() {
    int a;
    clrscr();
    a = 1;
    while(a <= 5) {
        printf("MCMT \t");
        a += 1; // i.e. a = a + 1
    }
    getch();
}
```

Output:

MCMT MCMT MCMT MCMT MCMT



Explanation: The loop starts with $a = 1$ and continues as long as $a \leq 5$. In each iteration, it prints "MCMT" and increments a by 1. The loop runs exactly 5 times.



The do-while Loop



Similar to while statement, but the **expression is evaluated at the end** of each loop.

Syntax:

```
do {  
    statement(s);  
} while(test condition);
```

Key Features:

- ✓ Body of loop is executed at least once
- ✓ Condition is evaluated after execution

vs. while Loop:

- ✓ do-while is called exit-control loop
- ✓ while is called entry-control loop



✓ Loop repeats as long as condition is true

✓ do-while guarantees at least one execution



do-while Loop Example

```
#include<stdio.h>
#include<conio.h>
void main() {
    int a;
    clrscr();
    a = 1;
    do {
        printf("MCMT\t"); // 5 times
        a += 1; // i.e. a = a + 1
    } while(a <= 5);
    getch();
}
```

Output:

MCMT MCMT MCMT MCMT MCMT



Explanation: The loop starts with $a = 1$, executes the body first, then checks if $a \leq 5$. It prints "MCMT" and increments a by 1 in each iteration. The loop runs exactly 5 times.



Infinite Loops



A looping process includes four steps. If the test condition **never becomes false**, an infinite loop occurs.

Looping Process Steps:

- 1 Setting of a counter
- 2 Execution of statements in the loop
- 3 Testing of a condition for loop execution
- 4 Incrementing the counter

Avoiding Infinite Loops:

- ✓ Ensure test condition will eventually be false
- ✓ Include proper counter increment
- ✓ Use Ctrl + C or Ctrl + Break to terminate

```
#include<stdio.h>
void main() {
    int i = 1;
    while(i <= 10) {
        printf("i = %d\n", i);
        // Missing increment operation (i++)
    }
}
```

```
}  
}
```



Problem: This program will never terminate as variable `i` will always be less than 10. To fix, add increment operation (`i++`) in the loop.



Summary



while Loop

Entry-control loop that checks condition before execution



do-while Loop

Exit-control loop that guarantees at least one execution



for Loop

Compact structure for fixed number of iterations



Remember to include proper counter increment to avoid infinite loops!

